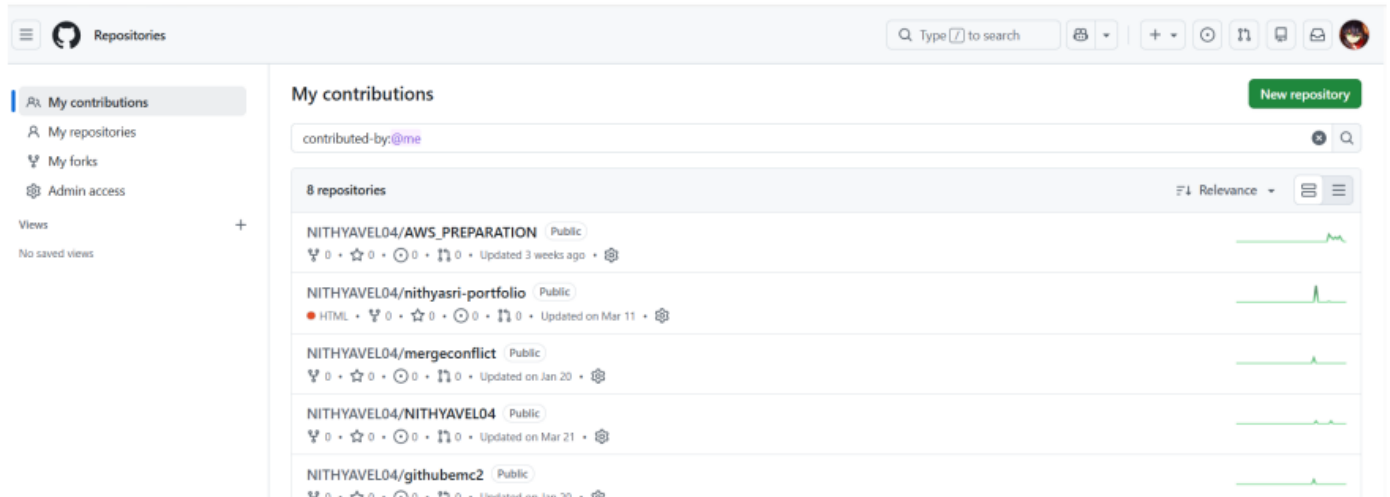


Git weekly test

1.Basic git commands

Hands on: Creating a repository in gitHub and implementing basic git commands

Step no: 1 Login to your GitHub account → Choose **New repository**.



Name: mynewrepoforpracticegit

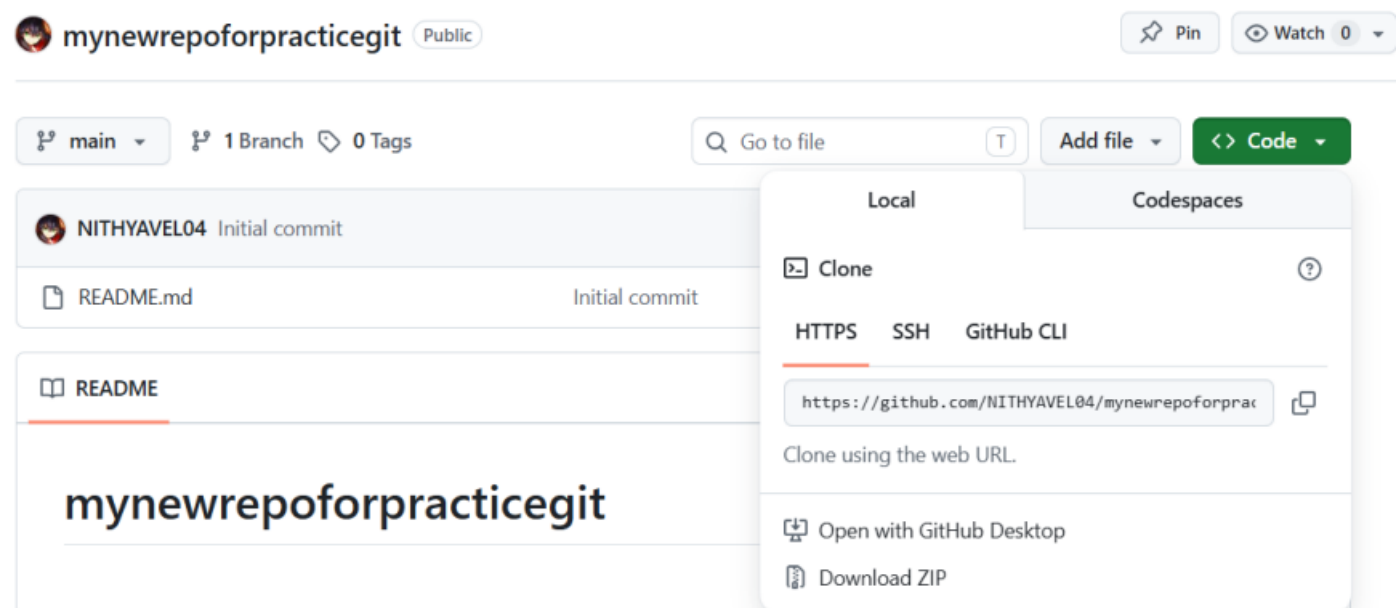
Type: Public

Enable: Readme

(.git ignore is used to store files that doesn't requires any tracking. So don't enable it.)

Create repository.

Once created → click **code** → copy the **https web url**



Step no: cloning the remote repo in local .

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~  
$ cd downloads  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ git clone https://github.com/NITHYAVEL04/mynewrepoforpracticegit.git  
Cloning into 'mynewrepoforpracticegit'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ cd mynewrepoforpracticegit  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)  
$ ls  
README.md  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)  
$ ls -la  
total 261  
drwxr-xr-x 1 nithya 197121  0 May 11 16:56 ./  
drwxr-xr-x 1 nithya 197121  0 May 11 16:56 ../  
drwxr-xr-x 1 nithya 197121  0 May 11 16:56 .git/  
-rw-r--r-- 1 nithya 197121 25 May 11 16:56 README.md
```

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)  
$ touch app1.py  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)  
$ ls  
README.md  app1.py
```

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)  
$ vim app1.py  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)  
$ cat app1.py  
print("hello world")
```

Creating a new file.

Checking the status

Adding to staging area followed by commit and push

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)  
$ git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    app1.py  
  
nothing added to commit but untracked files present (use "git add" to track)
```

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)
$ git add appl.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)
$ git status -s
A  appl.py
```

We can check the status, it will ask to push this into the remote repo.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)
$ git commit -m "My first commit"
[main 4c7ee3f] My first commit
1 file changed, 1 insertion(+)
create mode 100644 appl.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
```

`git log` → Shows all the commits

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)
$ git log
commit 4c7ee3fe7306d2f2ba9e6bf9acfccf05db310a1b (HEAD -> main)
Author: NITHYAVEL04 <nithyavel04@gmail.com>
Date: Mon May 11 17:14:27 2026 +0530

    My first commit

commit 2dcblbb74fdab6ced5810563bc8ba707d463f702 (origin/main, origin/HEAD)
Author: Nithya Sri Palanivel <124020920+NITHYAVEL04@users.noreply.github.com>
Date: Mon May 11 16:49:46 2026 +0530

    Initial commit
```

`git log --oneline` → Shows all the commits in short hash format

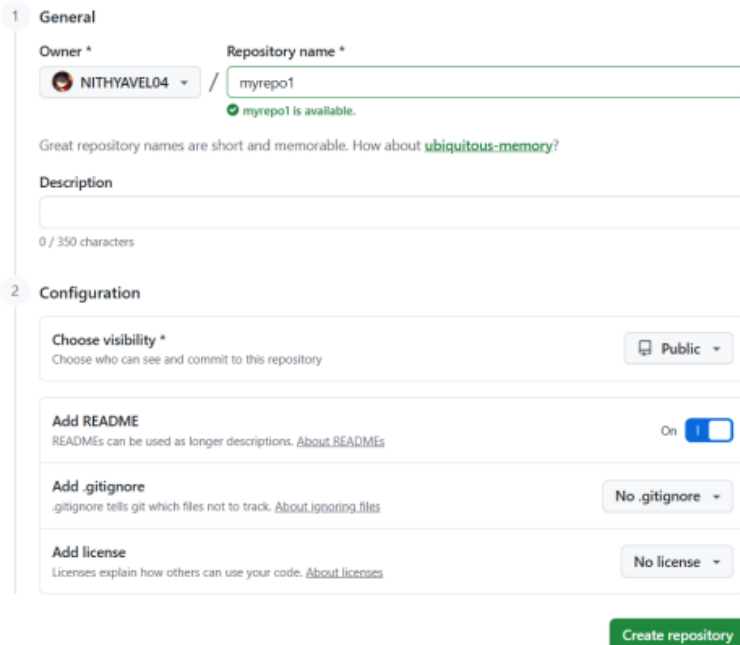
```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)
$ git log --oneline
4c7ee3f (HEAD -> main) My first commit
2dcblbb (origin/main, origin/HEAD) Initial commit
```

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/mynewrepoforpracticegit (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 308 bytes | 154.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NITHYAVEL04/mynewrepoforpracticegit.git
2dcblbb..4c7ee3f main -> main
```

2.Branch:

Hands on creating a repo with multiple branches

Step no:1 Create a remote repo in github (Keep it public and enable Readme file).



1 General

Owner *  NITHYAVEL04 / Repository name * myrepo1

myrepo1 is available.

Great repository names are short and memorable. How about [ubiquitous-memory](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *  Public

Choose who can see and commit to this repository

Add README

READMEs can be used as longer descriptions. [About READMEs](#) On ☒

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#) No license

Create repository

Step no:2 Copy the url and clone it in gitbash.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~  
$ cd downloads  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ git clone https://github.com/NITHYAVEL04/myrepo1.git  
Cloning into 'myrepo1'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.
```

Get inside of our repo

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ cd myrepo1/  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)  
$ |
```

Create a new file inside it.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ cd myrepo1/  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)  
$ vi myfile1.com
```

Step no:3 To see the available branches: `git branch`

The symbol * denotes the current branch.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git branch
* main
```

Step no:4 We are going to create a new branch called “dev”

`git branch dev`

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git branch dev
```

To switch to the dev branch: `git switch dev`

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git switch dev
Switched to branch 'dev'

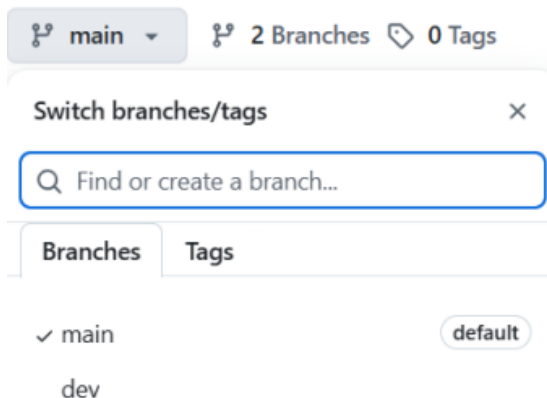
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ |
```

Push the changes.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git checkout dev
Switched to branch 'dev'

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git push origin dev
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
```

If we go to GitHub, we can see all the branches has been visible there.

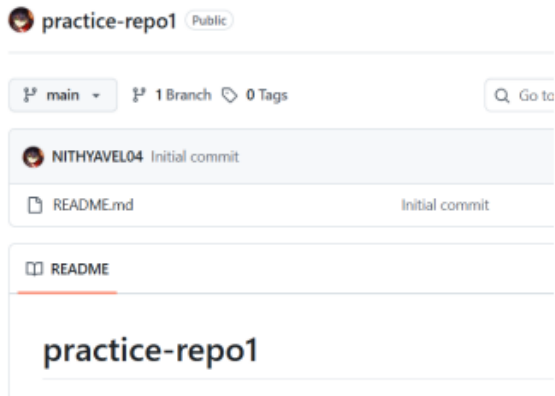


3.Merge and Pull request:

Hands on implementation of Merge & Pull requests

(Note : Both are same. Merge will be done in the CLI while Pull will be done in the GitHub console).

Step no:1 Create a remote repository in the git hub



Step no:2 Now open the git bash create a folder and clone the repository inside it.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ git clone https://github.com/NITHYAVEL04/practice-repo1.git
Cloning into 'practice-repo1'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (f
Receiving objects: 100% (3/3), done.

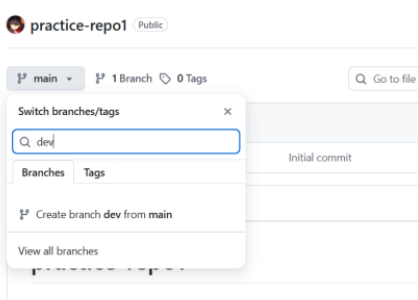
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ ls
practice-repo1/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ cd practice-repo1/
```

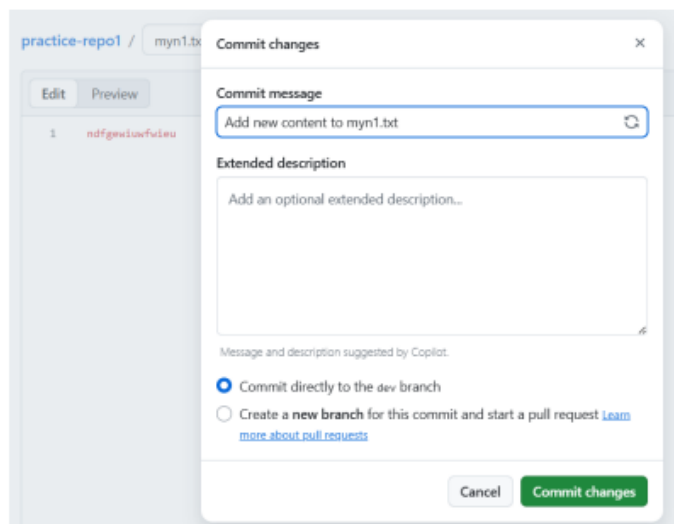
Step no:3 The git clone is used to bring the remote repo to the local, while the git pull is to bring the updates and changes made in the remote repo to the local.

Create a branch named: **dev** from **Main**.

Be aware of where we are creating the branch.



Step no:4 Now open the **dev** branch and add a file in it. Commit changes.



Step no:5 Now we are going to create a branch and merge it with main from cli.

So create a branch inside that repo from cli with same name of the branch in the git Hub.

Then switch to the branch

Once switched, pull the branch

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ cd practice-repo1/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ ls
README.md

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git branch dev

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git switch dev
Switched to branch 'dev'

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ git pull origin dev
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 962 bytes | 64.00 KiB/s, done.
From https://github.com/NITHYAVEL04/practice-repo1
 * branch          dev          -> FETCH_HEAD
 * [new branch]    dev          -> origin/dev
Updating 6c47028..c1931df
Fast-forward
 myn1.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 myn1.txt

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ |
```

First move to the main branch. (The branch must be where we are going to merge our changes, here I am using main)

Now merge our branch (dev) with the main.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git merge dev
Updating 6c47028..c1931df
Fast-forward
 myn1.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 myn1.txt

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ |
```

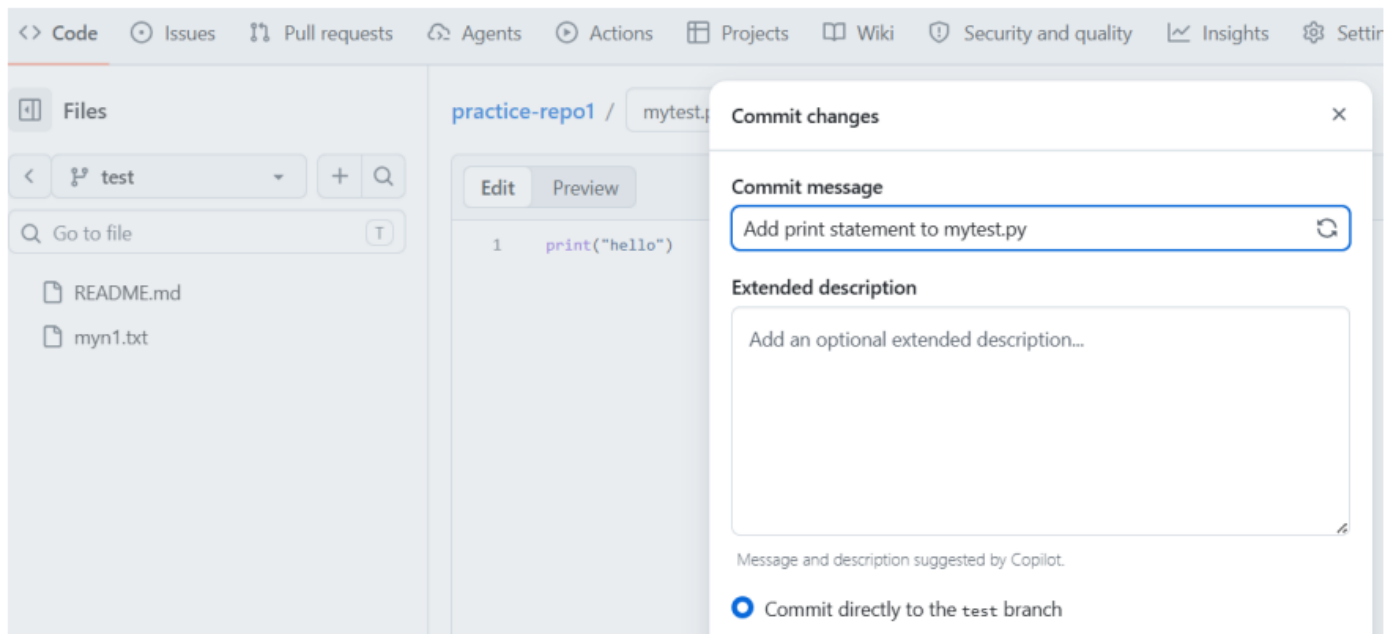
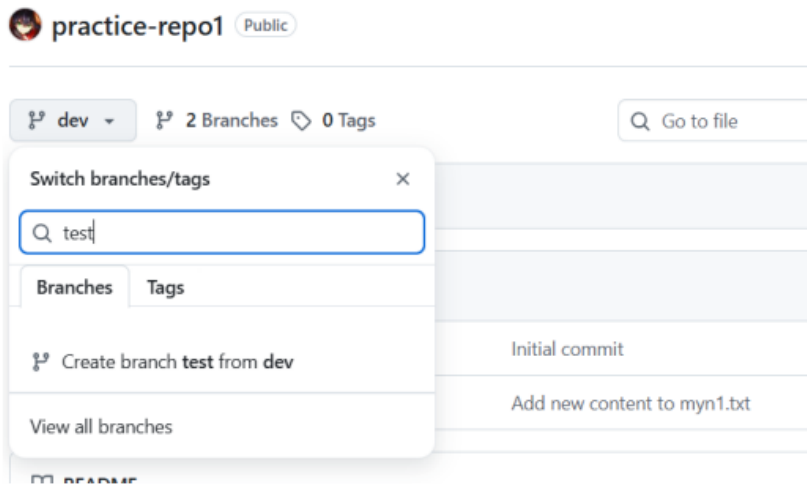
We can see both the commits in commit history of main as it is merged with dev locally.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git log --oneline
c1931df (HEAD -> main, origin/dev, dev) Add new content to myn1.txt
6c47028 (origin/main, origin/HEAD) Initial commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$
```


Step no:7 Now we are going to create a pull request in the git Hub.

Create a new branch from dev and add a file. Commit changes.



4.Git init

Hands on: Creating a repo in local using git init and add a remote repo to the local one:

Step no:01 Create a separate folder: **nithyafolder** in downloads.

Initialise .git folder in the local repo using: **git init**

Create a new file using **touch**.

Add it and make first commit.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~
$ cd downloads

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads
$ mkdir nithyafolder

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads
$ cd nithyafolder/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder
$ git init
Initialized empty Git repository in C:/Users/nithya/Downloads/nithyafolder/.git/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ touch app.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -m "first commit"
[main (root-commit) bf14698] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 app.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$
```

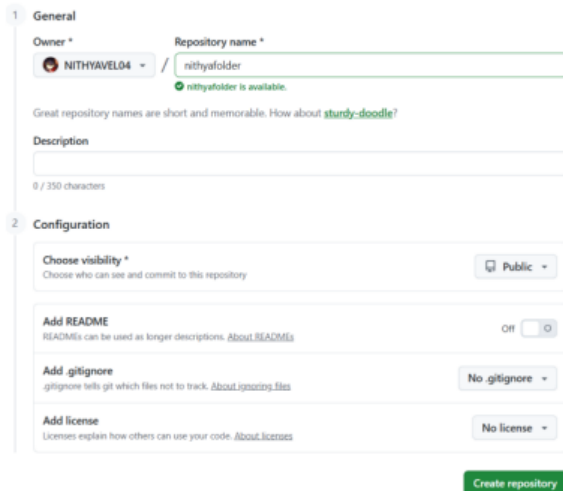
Rename the branch name from **master** to **main** : **git branch -M newname**

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git branch -M main
```

Step no:2 As we don't have any repository in remote to push this folder. Go to git hub and create a repo with same name as the folder name.

Ensure we are not making any changes, keep it default or else it will create a initial commit.

So just create a repo in remote without any changes.



The screenshot shows the GitHub repository creation interface. It is divided into two sections: 'General' and 'Configuration'. In the 'General' section, the 'Owner' is 'NITHYAVEL04' and the 'Repository name' is 'nithyafolder', with a green checkmark indicating it is available. There is a suggestion for 'sturdy-doodle?'. The 'Description' field is empty. In the 'Configuration' section, 'Choose visibility' is set to 'Public', 'Add README' is 'Off', 'Add .gitignore' is 'No .gitignore', and 'Add license' is 'No license'. A green 'Create repository' button is at the bottom.

Copy the repo url → git bash → add the remote origin to our local → push the local repo to that origin

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git remote add origin https://github.com/NITHYAVEL04/nithyafolder.git

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 219 bytes | 36.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NITHYAVEL04/nithyafolder.git
 * [new branch]      main -> main
```

Check the remote repo available we can see the link of our remote repo here.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git remote -v
origin https://github.com/NITHYAVEL04/nithyafolder.git (fetch)
origin https://github.com/NITHYAVEL04/nithyafolder.git (push)
```

5. Git Reset:

Using **reflog** we can see all the changes in the commit.

Now I am going to reset using soft followed by the commit id.

Soft only moves the head to previous commit.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reflog --oneline
2fa7232 (HEAD -> main) HEAD@{0}: commit: seventh commit
d6eb238 HEAD@{1}: commit: Sixth commit
c5cb36f HEAD@{2}: commit: fifth commit
3732bde HEAD@{3}: commit: fourth commit
a185a80 HEAD@{4}: commit: third commit
460f410 HEAD@{5}: commit: second commit
bf14698 (origin/main) HEAD@{6}: Branch: renamed refs/heads/main to refs/heads/main
bf14698 (origin/main) HEAD@{8}: commit (initial): first commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reset --soft 2fa7232

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git status -s

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reflog --oneline
2fa7232 (HEAD -> main) HEAD@{0}: reset: moving to 2fa7232
2fa7232 (HEAD -> main) HEAD@{1}: commit: seventh commit
d6eb238 HEAD@{2}: commit: Sixth commit
c5cb36f HEAD@{3}: commit: fifth commit
3732bde HEAD@{4}: commit: fourth commit
a185a80 HEAD@{5}: commit: third commit
460f410 HEAD@{6}: commit: second commit
bf14698 (origin/main) HEAD@{7}: Branch: renamed refs/heads/main to refs/heads/main
bf14698 (origin/main) HEAD@{9}: commit (initial): first commit
```

Using mixed it moves the HEAD to the specified commit and updates the index to match, but does not change the working directory. This is the default mode.

It will delete the previous commit, like if we have any tracked files which has not been pushed, it will be role backed to untracked files.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reset --mixed 2fa7232

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reflog --oneline
2fa7232 (HEAD -> main) HEAD@{0}: reset: moving to 2fa7232
2fa7232 (HEAD -> main) HEAD@{1}: reset: moving to 2fa7232
2fa7232 (HEAD -> main) HEAD@{2}: commit: seventh commit
d6eb238 HEAD@{3}: commit: Sixth commit
c5cb36f HEAD@{4}: commit: fifth commit
3732bde HEAD@{5}: commit: fourth commit
a185a80 HEAD@{6}: commit: third commit
460f410 HEAD@{7}: commit: second commit
bf14698 (origin/main) HEAD@{8}: Branch: renamed refs/heads/main to refs/heads/m
bf14698 (origin/main) HEAD@{10}: commit (initial): first commit
```

6.Git Fork:

Significant-Gravitas / AutoGPT

Issues 258 Pull requests 149 Agents Actions Wiki Security and quality 21 Insights

AutoGPT Public Watch 1532 Fork 46.2k

master 759 Branches 106 Tags

Go to file Add file <> Code

Local Codespaces

Clone

HTTPS SSH GitHub CLI

https://github.com/Significant-Gravitas/AutoGPT

Clone using the web URL

Open with GitHub Desktop

Download ZIP

About

AutoGPT is the vision for accessible AI for everyone, to use and to build on. Our mission is to provide the tools, that you can focus on your own ideas.

agpt.co

python ai art openai gpt autonomous-agents llama-api agentic

Readme View license

Create a new fork

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. [View existing forks.](#)

Required fields are marked with an asterisk (*).

Owner * Repository name *

NITHYAVEL04 / AutoGPT

AutoGPT is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

AutoGPT is the vision of accessible AI for everyone, to use and to build on. Our mission is to provide the tools, 152 / 350 characters

☒ Copy the master branch only

Contribute back to Significant-Gravitas/AutoGPT by adding your own branches. [Learn more.](#)

You are creating a fork in your personal account.

Create fork